PROJET PALETTISEUR

OSTR

Bilel ZAKANI-FADILI

Système embarqué
SE4 - Promotion 2024-2027

Professeur référent : Mr. Laurent LATORRE

Table des matières

1. INTRODUCTION 5

2. ARCHITECTURE FONCTIONNELLE TEMPS RÉEL 6

2.1. Architecture Globale 6

2.2. Tâches 9

2.3. Stratégies 14

3. RÉSULTATS 16

3.1. Rendu du palettiseur 16

3.2. Gestion de la mémoire 18

3.3. Utilisation du CPU 20

3.4. Consommation énergétique 21

4. CONCLUSION 24

Table des illustrations

Figure 1 : Architecture du projet Palettiseur 6
Figure 2 : Format de la trame de communication avec le bridge Python 6
Figure 3 : Adressage des actionneurs et capteurs dans le code C 7
Figure 4 : Modification de la fonction printchar.c 8
Figure 5 : Structures de communications 9
Figure 6 : Architecture du STM32 10
Figure 7 : Vue d'ensemble du Palettiseur en simulation 16
Figure 8 : Partie 1 du schéma fonctionnel du Palettiseur 17
Figure 9 : Partie 2 du schema fonctionnel du Palettiseur 17
Figure 10 : High Water Marks des tâches 18
Figure 11 : Mapping mémoire de la carte nucleo 18
Figure 12 : High Water Marks des taches après redimensionnement de celles-ci 19
Figure 13 : Consommation initiale du Palettiseur 21
Figure 14 : Modification de la fonction d'initialisation de la liaison UART1 22
Figure 15 : Consommation du Palettiseur après modification de la fréquence CPU à 8 MHz 22
Figure 16 : Modification au sein du fichier FreeRTOSConfig.h 23
Figure 17 : Fonction d'exécution dans l'état IDLE 23
Figure 18 : Consommation finale du Palettiseur 23

Table des références

Tableau 1 : Connectivité UART de la carte nucleo 8
Tableau 2 : Utilisation CPU des tâches 20

1. INTRODUCTION

Dans ce projet, Il nous est proposé de mettre en œuvre le contrôle temps-réel d'un système indutriel. Le système industriel considéré est le « Palettiseur », simulé dans l'environnement RealGames Factory I/O.

Ce document présente l'architecture, les observations ainsi que les points d'améliorations du système.

2. ARCHITECTURE FONCTIONNELLE TEMPS RÉEL

2.1. Architecture Globale

Le palettiseur est mis en œuvre à l'aide d'une carte d'évaluation nucleo STM32F072RB avec une couche FreeRTOS. Les observations se font à l'aide de l'outil « Factory IO ».

L'architecture du projet est représentée sur la figure suivante :

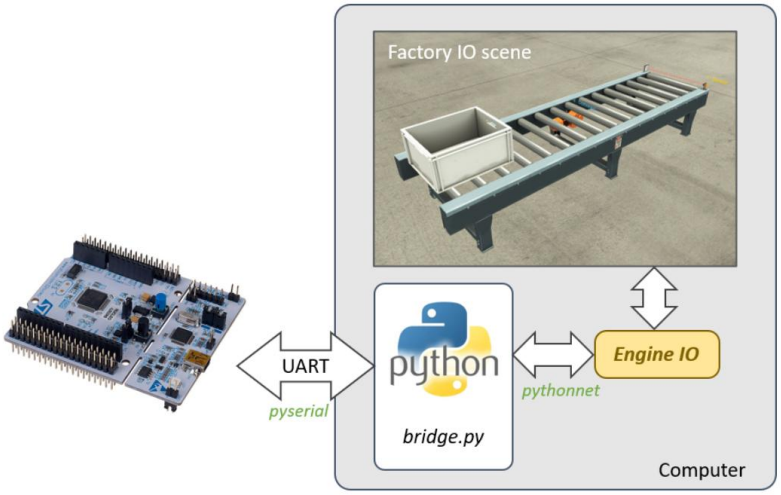


Figure 1 : Architecture du projet Palettiseur

L'interface entre la carte nucleo et Factory IO est réalisée à l'aide d'un bridge Python. Celui-ci réalise une communication avec la carte nucleo grâce à une interface UART à 115200 bauds.

Le bridge est par défaut configuré pour interpréter une trame de 8 octets de la forme suivante :

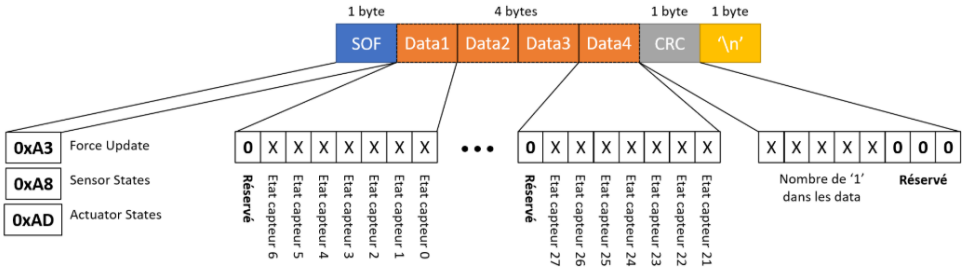


Figure 2 : Format de la trame de communication avec le bridge Python

Le bridge Python interprète la trame précédente si le premier octet est l'un des trois octets attendu (0xA3, 0xA8 ou 0xAD). Dans le cas contraire, le bridge réalise un affichage dans sa propre console.

Pour mettre à jour des actionneurs ou récupérer l'état des capteurs, Factory I/O met à disposition un adressage des actionneurs et des capteurs. Ces adresses sont utilisées dans la trame précédente envoyée au bridge.

De cette manière, chaque actionneur et chaque capteur possèdent une adresse précise spécifiée dans le code du STM32 :

```c
//Define actuators
#define Distribution_cartons            (uint32_t)(1)
#define Tapis_distrib_cartons           (uint32_t)(1 << 1)
#define Blocage_entree_palettiseur      (uint32_t)(1 << 2)
#define Porte                           (uint32_t)(1 << 3)
#define Poussoir                        (uint32_t)(1 << 4)
#define Clamp                           (uint32_t)(1 << 5)
#define Monter_ascenseur                (uint32_t)(1 << 6)
#define Descendre_ascenseur             (uint32_t)(1 << 8)
#define Ascenseur_to_limit              (uint32_t)(1 << 9)
#define Distribution_palette            (uint32_t)(1 << 10)
#define Charger_palette                 (uint32_t)(1 << 11)
#define Tapis_carton_vers_palettiseur   (uint32_t)(1 << 12)
#define Tourner_carton                  (uint32_t)(1 << 13)
#define Decharger_palettiseur           (uint32_t)(1 << 15)
#define Tapis_Charger_palettiseur       (uint32_t)(1 << 16)
#define Tapis_decharger_palette         (uint32_t)(1 << 17)
#define Tapis_Palette_Vers_Ascenseur    (uint32_t)(1 << 18)
#define Tapis_Distribution_Palette      (uint32_t)(1 << 19)
#define Tapis_Fin                       (uint32_t)(1 << 20)
#define Remover                         (uint32_t)(1 << 21)
```

```c
//Define sensors
#define Carton_distribue                (uint32_t)(1)
#define Carton_envoye                   (uint32_t)(1 << 1)
#define Entree_palletiseur              (uint32_t)(1 << 2)
#define Porte_ouverte                   (uint32_t)(1 << 3)
#define Limite_poussoir                 (uint32_t)(1 << 4)
#define Clamped                         (uint32_t)(1 << 5)
#define Ascenseur_etage_RDC             (uint32_t)(1 << 6)
#define Ascenseur_etage1                (uint32_t)(1 << 8)
#define Ascenseur_etage2                (uint32_t)(1 << 9)
#define Sortie_palette                  (uint32_t)(1 << 10)
#define Limite_porte                    (uint32_t)(1 << 11)
#define Ascenseur_en_mvt                (uint32_t)(1 << 12)
#define Entree_palette                  (uint32_t)(1 << 13)
```

Figure 3 : Adressage des actionneurs et capteurs dans le code C

L'adressage dans le code réalise un saut de bit tous les 7 bits car ce sont des bits réservés dans la trame.

Par défaut, les divers tapis sont activés de façon à ne pas les gérer ultérieurement.

Dans la suite du projet, il est décidé de mettre en place un affichage statique en console. Il est nécessaire pour cela d'utiliser des caractères d'échappement. Le bridge Python ne prend pas en charge les caractères d'échappement donc un deuxième UART est utilisé pour permettre l'affichage en console de texte.

De cette façon, l'affichage en console peut être effectuer par la fonction **my_printf** en reconfigurant la fonction **printchar** sur le deuxième UART :

```c
static void printchar(char **str, int c)
{
    if (str) {
        **str = c;
        ++(*str);
    }
    else
    {
        while ( (USART1->ISR & USART_ISR_TC) != USART_ISR_TC);
        USART1->TDR = c;
    }
}
```

Figure 4 : Modification de la fonction printchar.c

L'ensemble de la connectivité UART de la carte nucleo est définie dans le tableau suivant :

Pin	Fonction	Channel DMA	Baudrate	Description
UART connecté au bridge Python				
PA2	USART2_TX	Channel 4	115200	Pin TX de la liaison UART
PA3	USART2_RX			Pin RX de la liaison UART
UART d'affichage en console				
PA9	USART1_TX	-	115200	Pin TX de la liaison UART

Tableau 1 : Connectivité UART de la carte nucleo

Le mode DMA est utilisé pour l'UART 2 connecté au bridge pour optimiser l'utilisation du CPU.

L'occupation CPU par la fonction **my_printf** est négligé donc le DMA n'est pas utilisé pour l'UART 1.

Pour étudier les performances du projet, l'outil Tracealyzer 4 de Percepeio est utilisé. Le palettiseur se déroulant sur plusieurs secondes voire minutes, le mode streaming est mis en place pour ne pas avoir de contraintes sur l'acquisition des données.

L'objectif global du Palettiseur est de construire une palette de deux couches de six cartons chacune. Plusieurs tâches doivent pouvoir s'exécuter en parallèles.

2.2. Tâches

L'architecture du système est divisée en plusieurs tâches réalisant des actions spécifiques.

Parmi elles, deux tâches réalisent le codage et décodage pour l'interface de communication : « Write » et « Publish »

L'interface entre les tâches communes et les tâches de communication (Write et Publish) est mis en œuvre par deux structures.

La structure « message_t » permet aux tâches communes de spécifier les actionneurs à activer ou désactiver.

La structure « subscribe_message_t » permet aux tâches communes d'être alerté par l'état d'un capteur via un sémaphore.

```
typedef struct
{
    uint32_t mask;              // Actionner ID
    uint32_t state;             // Actionner State
} message_t;
```

```
typedef struct
{
    uint32_t sensor_mask;       // Awaited sensor ID
    uint32_t sensor_state;      // Awaited sensor State
    xSemaphoreHandle *xSem;     // Semaphore linked to the response
} subscribe_message_t;
```

Figure 5 : Structures de communications

<u>La tâche « Write » :</u>

La tâche Write reçoit des messages des différentes tâches du contrôleur et a pour mission de mettre à jour l'état des actionneurs, aussi rapidement que possible. Elle réalise la gestion des messages à l'aide d'une « Message Queue » et met en forme le message pour effectuer une transaction UART via DMA.

Dans le code, deux variables globales servent à compter :

- Le nombre total de cartons expédié depuis le début
- Le nombre de palette envoyée

Ces deux variables sont ensuite affichées en console.

<u>La tâche « Publish » :</u>

La tâche Publish lit périodiquement l'état des capteurs et « publie » cette information afin que les tâches concernées puissent réagir au besoin. La tâche Publish est donc dans ce modèle le « publisher ». La gestion des messages est réalisée au travers d'une « Message Queue ». Chaque message reçu est stocké dans un tableau pouvant accueillir 5 messages. L'état des capteurs est recueilli par le buffer UART puis comparé à chaque message stocké dans le tableau. Si l'un des capteurs possède l'état attendu par un message, l'information est traitée par la libération du sémaphore associé au message et l'emplacement du tableau est libéré.

Ces deux tâches servent d'intermédiaire entres les tâches exécutants des actions et l'interface de communication.

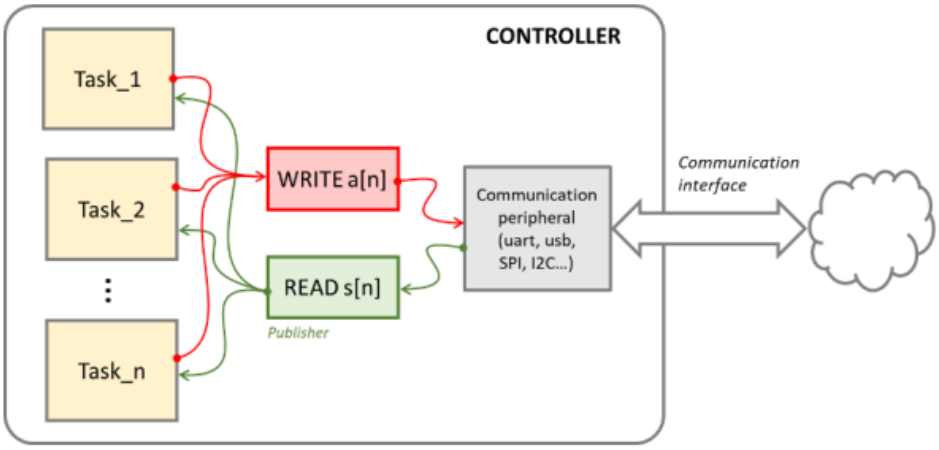


Figure 6 : Architecture du STM32

La décomposition du système en tâches réalisant une action ou groupe d'actions permet des exécutions parallèles de manière simplifiée. Le palettiseur est mis en œuvre via les tâches suivantes :

- Tâche « Carton »
- Tâche « Barriere »
- Tâche « Poussoir and porte »
- Tâche « Ascenseur »
- Tâche « Palette »

La tâche « Carton » :

Cette tâche réalise la gestion de l'envoi des cartons. A la suite de la réception d'une notification de la tâche « barriere », cette tâche envoi deux cartons à la suite.

La tâche « Barriere » :

Cette tâche réalise la gestion de la barrière de cartons. Elle réalise des actions séquentielles qui sont les suivantes :

1. Attente d'un sémaphore libéré par la tâche « Poussoir and porte » : xSem_porte_ready
2. Active le bloqueur
3. Envoi d'une notification à la tâche « Carton »
4. Attente de l'arrivée du deuxième carton au niveau du bloqueur
5. Attente d'un sémaphore libéré par la tâche « Poussoir and porte » : xSem_poussoir_ready
6. Libération d'un sémaphore à la tâche « Poussoir and porte » : xSem_cartons_bloque
7. Désactivation du tapis de distribution de cartons
8. Désactivation du bloqueur
9. Attente de 700 ms pour que les cartons traversent le bloqueur
10. Réactivation du tapis de distribution de cartons

Une désactivation temporaire du tapis de distribution autour de la désactivation du bloqueur est réalisée pour éviter que certains cartons tournent sous l'effet de l'abaissement de la barrière.

<u>La tâche « Poussoir and porte » :</u>

Cette tâche réalise la gestion du poussoir et de la porte. Elle réalise les actions séquentielles qui sont les suivantes :

1. Attente d'un sémaphore libéré par la tâche « Barriere » : xSem_cartons_bloque
2. Attente de 3,5 s le temps que les cartons arrivent en butée
3. Activation du poussoir
4. Attente de 500 ms que le poussoir soit en mouvement
5. Attente de l'arrivée du poussoir au bout
6. Désactivation du poussoir
7. Attente du retour complet du poussoir
8. Libération d'un sémaphore à la tâche « Barriere » : xSem_poussoir_ready
9. Incrémente le compteur
10. Libération d'un sémaphore à la tâche « Barriere » : xSem_porte_ready

Tant que le compteur est inférieur à 3, les actions précédentes sont exécutées.

Lorsque le compteur est égal à 3 :

1. Attente de la notification par la tâche « Ascenseur »
2. Activation du clamp
3. Attente fin du serrage
4. Ouverture de la porte
5. Attente fin ouverture de la porte
6. Relâchement du clamp
7. Envoi d'une notification à la tâche « Ascenseur »
8. Attente de 2 s pour que l'ascenseur puisse descendre s'il l'est à l'étage 1
9. Fermeture porte
10. Attente fermeture porte
11. Réinitialisation du compteur

La tâche « Ascenseur » :

Cette tâche réalise la gestion de l'Ascenseur. Elle réalise les actions séquentielles qui sont les suivantes :

1. Attente détection de la palette en sortie d'ascenseur
2. Désactivation du tapis de l'ascenseur
3. Élévation de l'ascenseur à l'étage 1
4. Attente fin de mouvement de l'ascenseur
5. Vérification de la position de l'ascenseur à l'étage 1
6. Désactivation du mouvement jusqu'à la limite
7. Envoie d'une notification à la tâche « Poussoir and porte »
8. Attente de la notification par la tâche « Poussoir and porte »
9. Descente de l'ascenseur à l'étage 2
10. Attente fin de mouvement de l'ascenseur
11. Envoie d'une notification à la tâche « Poussoir and porte »
12. Attente de la notification par la tâche « Poussoir and porte »
13. Descente de l'ascenseur au RDC
14. Attente de l'arrivée de l'ascenseur au RDC
15. Désactivation de la descente de l'ascenseur
16. Activation du tapis de l'ascenseur
17. Attente de la sortie de la palette depuis l'ascenseur
18. Envoie d'une notification à la tâche « Palette »

La tâche Palette :

Cette tâche réalise la gestion de l'envoi des palettes. Elle réalise les actions séquentielles qui sont les suivantes :

1. Activation de la distribution de palette
2. Attente de 1 s
3. Désactivation de la distribution de palette
4. Attente de la notification par la tâche « Ascenseur »

Pour une meilleur robustesse du système et coordination des actions, des mécanismes d'interruption et divers objets kernel tels que des sémaphores et notifications sont utilisés.

La tâche « HWM » (High Water Mark) :

Cette tâche est purement destinée à du debug. Elle récupère la mémoire disponible de la stack de chaque tâche. Les tailles mémoires restantes sont ensuite affichées en console.

2.3. Stratégies

Les objets kernel ont été choisi afin d'optimiser la taille de la stack de chaque tâche. Par défaut, des sémaphores sont utilisés pour synchroniser des tâches entre elles. Si un sémaphore est libéré par une tâche et prise par une unique seconde tâche, alors le mécanisme de notification est utilisé.

Les sémaphores sont également utilisés pour le système de « Publish ».

Deux tâches réalisent de l'affichage en console (« Write » et « HWM »). Un mutex est utilisé pour éviter l'affichage simultanés de 2 messages.

Le niveau de priorité des tâches est choisi de sorte qu'une tâche ne soit préemptée que par une autre tâche réalisant une action plus importante.

Par exemple, les tâches « Write » et « Publish » ont la plus grande priorité car elles réalisent les mises à jour des actionneurs et des capteurs.

La tâche « Carton » est ensuite prioritaire de par sa contrainte d'activation et désactivation de la distribution de cartons.

La tâche « Ascenseur » est ensuite prioritaire car elle doit pouvoir détecter l'arrivée de la palette dans l'ascenseur.

Les tâches « Poussoir and Porte » et « Palette » ont la même priorité car elles nécessitent l'exécution des tâches précédentes.

La tâche « Barriere » est l'avant dernière moins prioritaire car elle nécessite l'exécution de la tâche « Poussoir and porte ».

La tâche « HWM » est la moins prioritaire car c'est une tâche de debug.

En résumé, le système de priorité des tâches est le suivant :

1. Tâche « HWM »
2. Tâche « Barriere »
3. Tâche « Poussoir and porte » et « Palette »
4. Tâche « Ascenseur »
5. Tâche « Carton »
6. Tâche « Publish »
7. Tâche « Write »

Concernant la mémoire, chaque tâche est initialisée avec une stack d'environ 150w (w=words) sauf la tâche « HWM » qui sert de debug et est initialisée au minimum : 128w. Le choix des tailles est purement arbitraire et sera par la suite redéfinit par les résultats fournit par la tâche « HWM ».

3. RÉSULTATS

3.1. Rendu du palettiseur

Une fois l'architecture du palettiseur créée, les résultats sont observés par l'application du protocole suivant :

1. Ouvrir le bridge Python et se connecter au port COM de la carte nucleo
2. Ouvrir la scène « Palletizer_1_4 » sur Factory IO et démarrer celle-ci
3. Démarrer le mode streaming de Tracealyzer4
4. Initialiser la séquence de debug du code via STM32CubeIDE

Une fois le protocole précédent suivi, le système Palettiseur est en marche et l'affichage en console est réalisé :

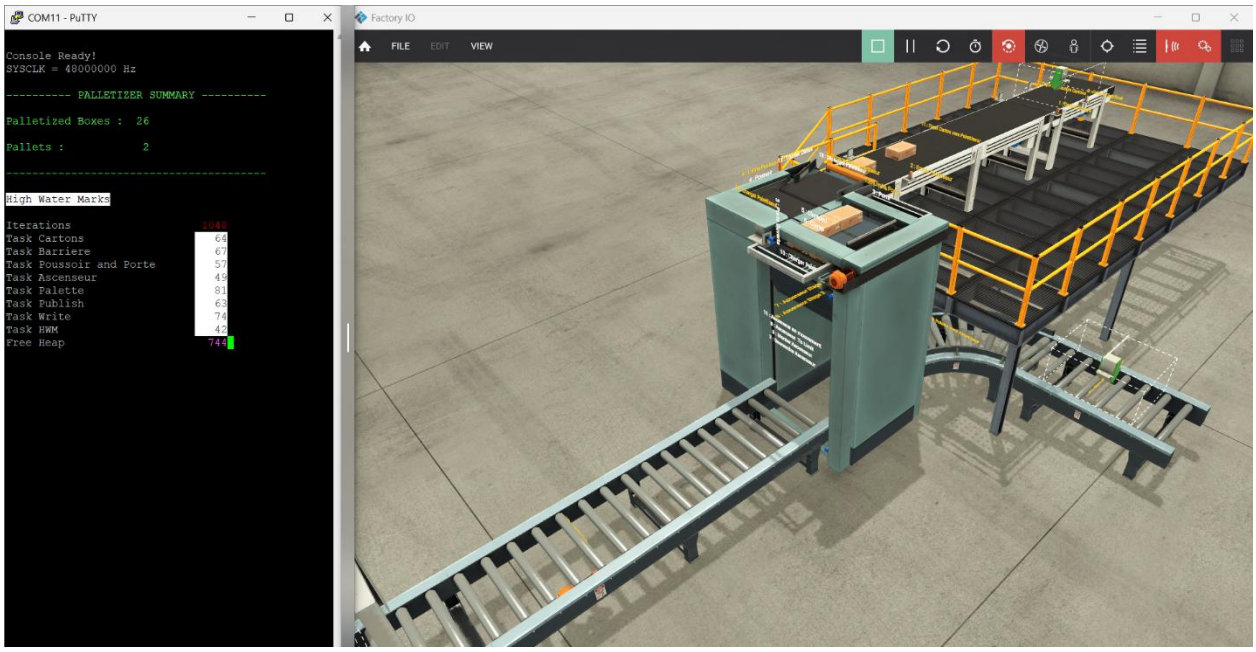


Figure 7 : Vue d'ensemble du Palettiseur en simulation

En capturant un cycle complet via Tracealyzer, le schéma fonctionnel du Palettiseur peut être extrait. Il montre les différentes relations entre les tâches, objets kernel, ou encore interruptions. Il apporte une vision simplifié du programme. Le schéma généré par Tracealyzer est le suivant :

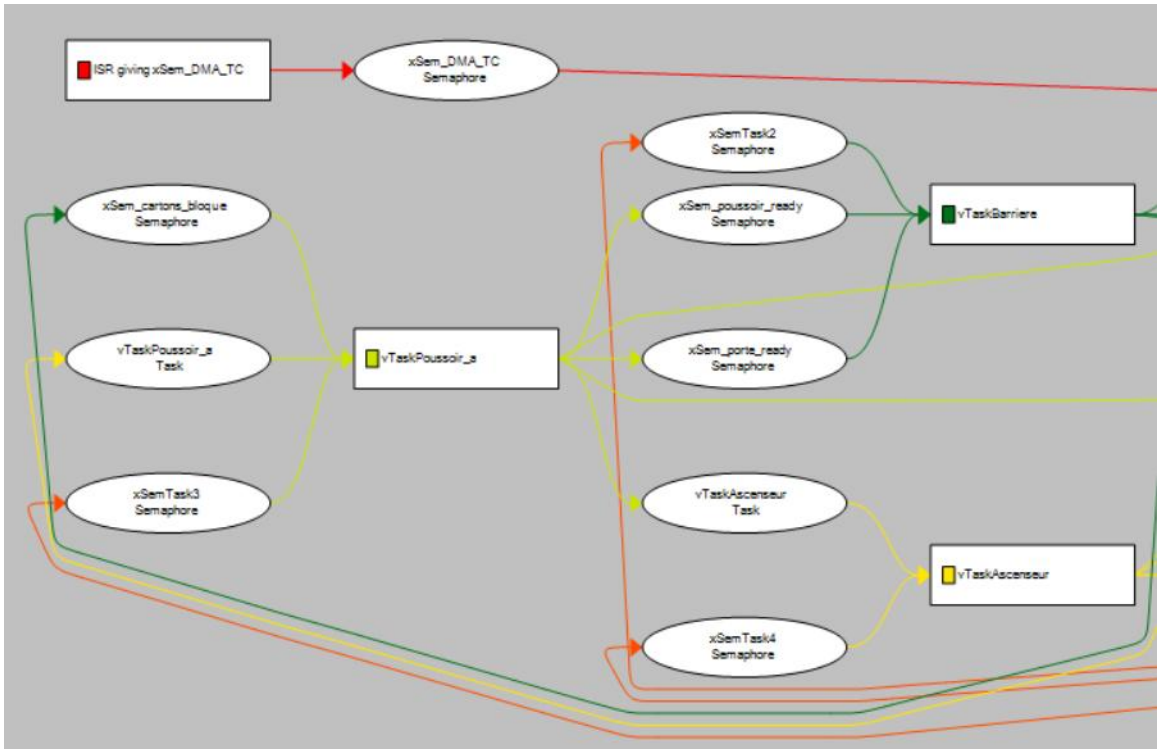


Figure 8 : Partie 1 du schéma fonctionnel du Palettiseur

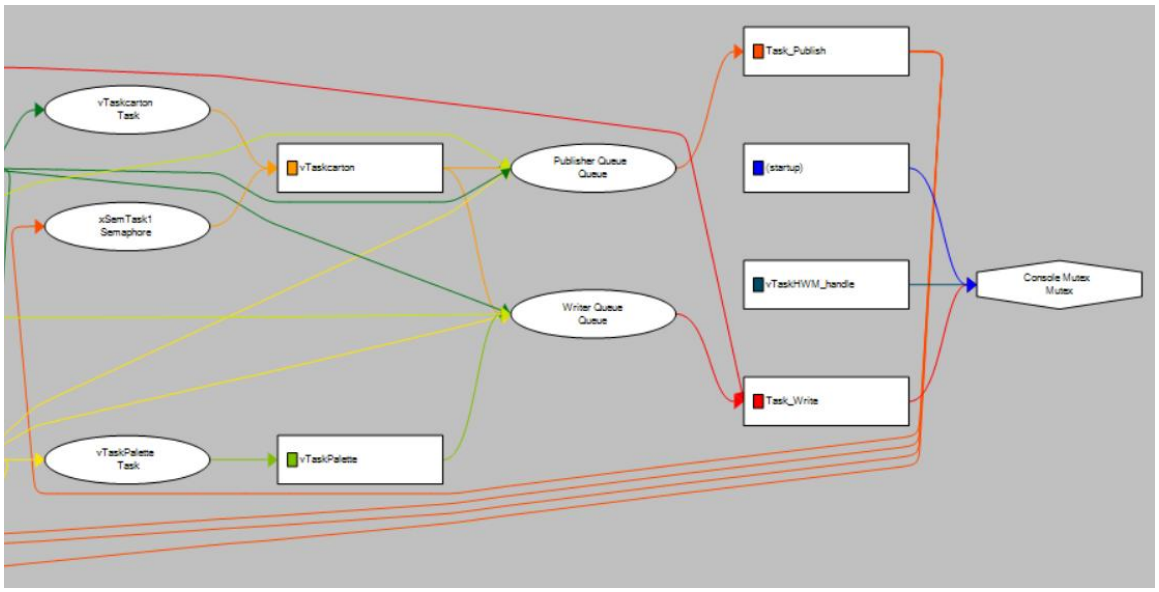


Figure 9 : Partie 2 du schema fonctionnel du Palettiseur

3.2. Gestion de la mémoire

La couche FreeRTOS du Palettiseur est configurée avec une heap de 9kB. Grâce à l'affichage réalisé par la tâche « HWM », les tailles restantes sur la stack de chaque tâche sont les suivantes :

```
High Water Marks

Iterations                    495
Task Cartons                   64
Task Barriere                  62
Task Poussoir and Porte        52
Task Ascenseur                 44
Task Palette                   77
Task Publish                   58
Task Write                     74
Task HWM                       42
Free Heap                     744
```

Figure 10 : High Water Marks des tâches

On observe également que la heap est presque saturée.

La taille de la RAM disponible sur le STM32 est de 88,13 % :

Memory Regions | Memory Details

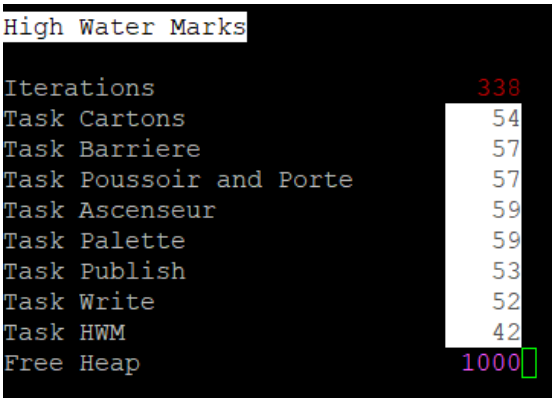
Region	Start address	End address	Size	Free	Used	Usage (%)
FLASH	0x08000000	0x0801ffff	128 KB	94,13 KB	33,87 KB	26,46%
RAM	0x20000000	0x20003fff	16 KB	1,9 KB	14,1 KB	88,13%
MEMORY...	0x60000000	0x5fffffff	0 B	0 B	0 B	

Figure 11 : Mapping mémoire de la carte nucleo

Chaque tâche devrait avoir une taille restante dans la stack d'environ 50w. Par conséquent, les stacks sont redéfinis :

- La tâche « Cartons » passe d'une stack de 150w à 140w
- La tâche « Barriere » passe d'une stack de 150w à 140w
- La tâche « Ascenseur » passe d'une stack de 150w à 160w
- La tâche « Palette » passe d'une stack de 150w à 128w
- La tâche « Publish » passe d'une stack de 150w à 140w
- La tâche « Write » passe d'une stack de 150w à 128w

Les tailles restantes de la stack de chaque tâche sont maintenant en moyenne autour de 50w :



High Water Marks	
Iterations	338
Task Cartons	54
Task Barriere	57
Task Poussoir and Porte	57
Task Ascenseur	59
Task Palette	59
Task Publish	53
Task Write	52
Task HWM	42
Free Heap	1000

Figure 12 : High Water Marks des taches après redimensionnement de celles-ci

La taille restante de la Heap est maintenant de 1000 octets mais aucune modification de la celle-ci n'est possible.

3.3. Utilisation du CPU

L'outil Tracealyzer4 permet également d'observer l'utilisation du CPU tout au long de l'enregistrement. L'utilisation CPU de chaque tâche est présentée dans le tableau suivant :

Tâche	Usage CPU (%)
Cartons	1,5e-5
Barriere	9,37e-4
Poussoir and Porte	1,4e-5
Ascenseur	0,036
Palette	1e-6
Write	0,247
Publish	1,26e-4
HWM	6,97
IDLE	92,7

Tableau 2 : Utilisation CPU des tâches

Le Palettiseur n'utilise presque pas le CPU car celui-ci est en état d'IDLE pendant près de 93% du temps.

L'utilisation du CPU dépends principalement des actions réalisés par les tâches ainsi que de leurs fréquences d'activation . La tâche « Palette » est ici la moins occupante car elle ne faite qu'envoyer une palette sur tout le cycle. En revanche, la tâche la plus occupante est « HWM » car elle réalise l'affichage en console à l'aide la fonction **my_printf**.

3.4. Consommation énergétique

La consommation de la carte est mesurée à l'aide d'un multimètre par le connecteur IDD de la carte.

En l'état actuel du projet, la consommation de la carte est de 16,13 mA :

Figure 13 : Consommation initiale du Palettiseur

Cette consommation peut être diminuée par un meilleur choix de la fréquence CPU.

En effet, le microcontrôleur est cadencé à 48 MHz et peut être abaisser à 8 MHz (fréquence par défaut) car le CPU est en majeure partie du temps dans l'état IDLE.

Le rétablissement de la fréquence CPU par défaut se fait par la mise en commentaire de la fonction **SystemClock_Config**.

Si la fréquence CPU évolue, la configuration des liaisons séries doit aussi être à jour. Plus précisément, c'est le registre BRR (BaudRate Register) qui doit être à jour.

A 48 MHz, le BRR calculé est de 833 (en activant le mode oversampling). A 8 MHz, le BRR devient égal à 139.

Cette modification est apportée au sein des fonctions d'initialisation des liaisons UART1 et UART2 :

```
// Baud Rate = 115200
// With OVER8=0 and Fck=8MHz, USARTDIV =   8E6/115200 = 69.4
// BRR = 69 -> Baud Rate = 115942.0289 -> 0.99% error
//
// With OVER8=1 and Fck=8MHz, USARTDIV = 2*8E6/115200 = 138.8
// BRR = 139 -> Baud Rate = 115108 -> 0.04% error (better)
RCC->CFGR3 |= RCC_CFGR3_USART1SW_SYSCLK;
USART1->CR1 |= USART_CR1_OVER8;
USART1->BRR = 139;
```

Figure 14 : Modification de la fonction d'initialisation de la liaison UART1

À la suite de cette modification, la consommation est passée de 16,13 mA à 4,8 mA :

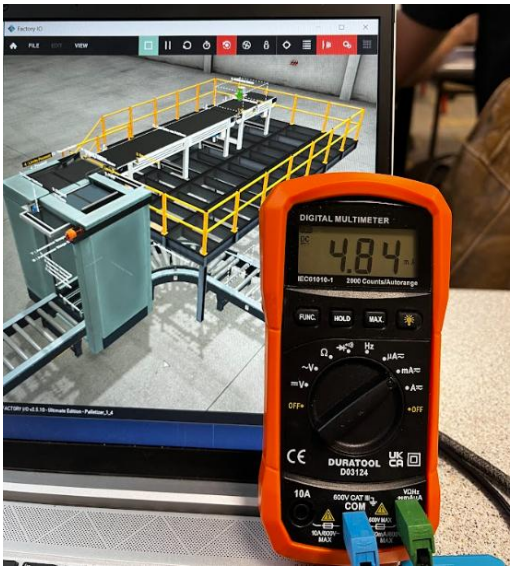


Figure 15 : Consommation du Palettiseur après modification de la fréquence CPU à 8 MHz

Cette consommation peut davantage être diminuée en configurant la couche FreeRTOS de sorte que la carte soit en mode low-power lorsque l'ordonnanceur est dans l'état IDLE.

Ce changement est réalisé par une modification dans les fichiers sources de FreeRTOS :

```
/* Hook function related definitions. */
#define configUSE_IDLE_HOOK                     1
```

Figure 16 : Modification au sein du fichier FreeRTOSConfig.h

Cette modification permet d'exécuter une fonction à chaque fois que l'ordonnanceur se trouve dans l'état IDLE. Cette fonction met le microcontrôleur en mode low-power :

```
void vApplicationIdleHook(void){

    // Go into Sleep mode
    __WFI();
}
```

Figure 17 : Fonction d'exécution dans l'état IDLE

A chaque tick OS, le mode low-power est interrompu pour vérifier si une tâche est prête à s'exécuter.

À la suite de ces modifications, la consommation finale de la carte devient 3,49 mA, soit une consommation divisée par 4,62 comparée à celle de départ :

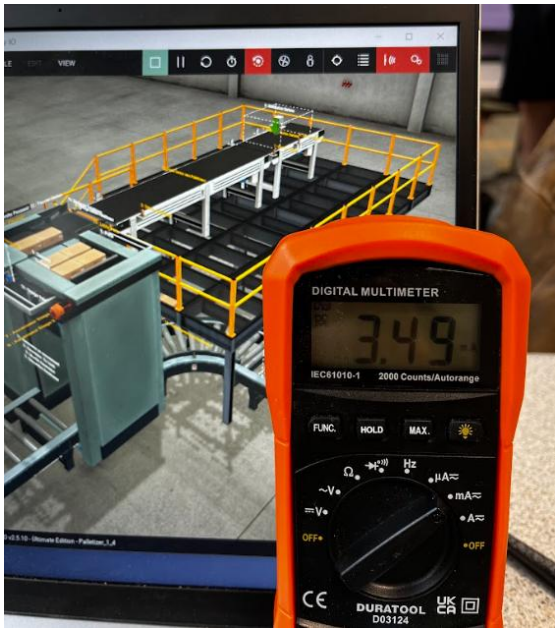


Figure 18 : Consommation finale du Palettiseur

4. CONCLUSION

Un système de contrôle temps-réel d'un Palettiseur est conçu en utilisant l'architecture FreeRTOS, le tout en simulant à l'aide de l'outil Factory IO.

Différentes méthodes et stratégies ont été mises en place pour garantir une certaine robustesse du système.

Les performances du systèmes ont été analysées à l'aide de l'outil Tracealyzer4 pour ensuite optimiser le système ou trouver ses limites.